Session 1

part_1 : Theory

the knowledge you need to be a Real Engineer!

Objectives

By the end of this session, You will be able to:

1. Understand what “backend” means and how it fits in a full-stack system.
2. Describe the role of a backend engineer.
3. Set up development environment (Node.js + JavaScript).
4. Write and run basic JavaScript code.
5. Understand variables, data types, and simple logic in JavaScript.

Don’t forget our Rules

1. 15 mins late is acceptable, after 15 mins every minute is equal to 1 pound. max(20 pounds)
2. Absence without any excuse -> 2 warnings
3. Missing Task without any excuse -> 1 warning
4. Any unacceptable behavior -> warning

Don’t forget our Rules

1. Any correct answer in the session = Mohsens
2. Best member of the week will be awarded next session
3. Best of the month will be awarded 1/12 based on mohsens
4. Best of the Season will be awarded at the end of the season based on mohsens and overall performance

Table of contents

01

Recap

02

Tour in our Resources

03

What is Backend?

04

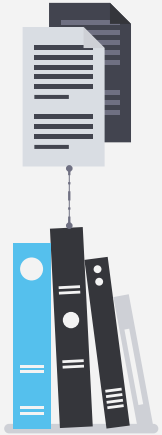
Backend Engineer Mindset

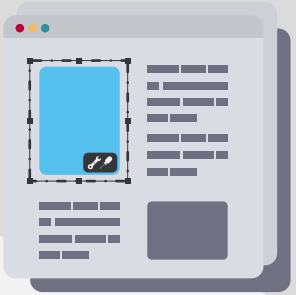
05

Languages & Tools Overview

06

JavaScript Deep Dive





01 Recap

- What happened last Session?
- What did we talk about?
- What was the benefit you got from last session?

Last Session Table of Contents

01

Student activities

What are they?
What they do?

02

What is OSC?

Who are we?
What is Open Source?
History of OSC

03

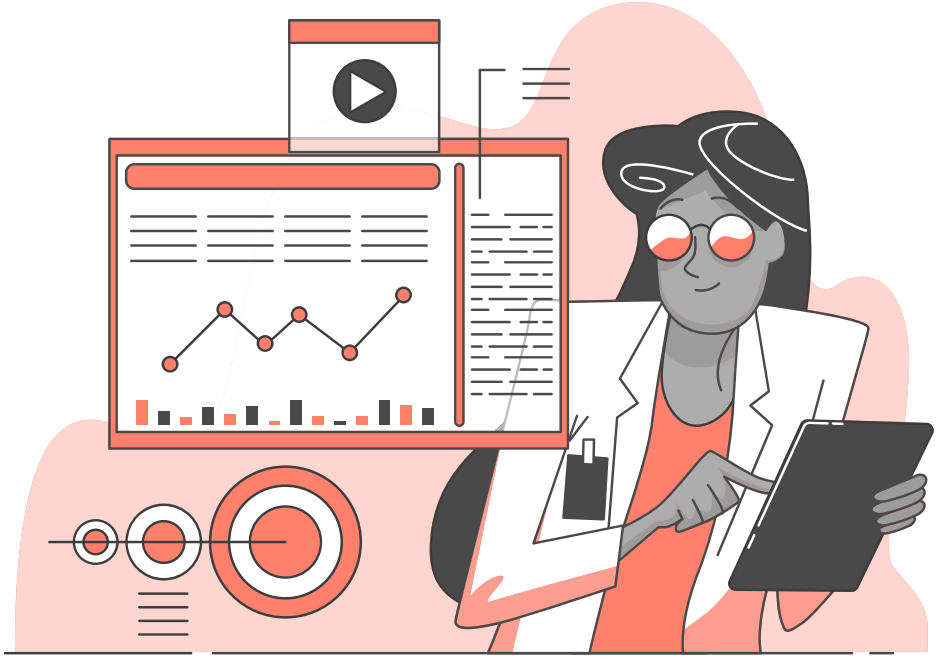
OSC Hierarchy

Who is the Board?
Who is the High Board?

04

Backend Committee

History
What do we do?
Committee Structure
Structure of each Session
Outline of each Level & Phases



Last Session Table of Contents

05

Backend Committee

Punishment System
Rewarding System
Reporting System

06

Backend

What is Backend?
Why Backend?
How to be Backend Engineer?

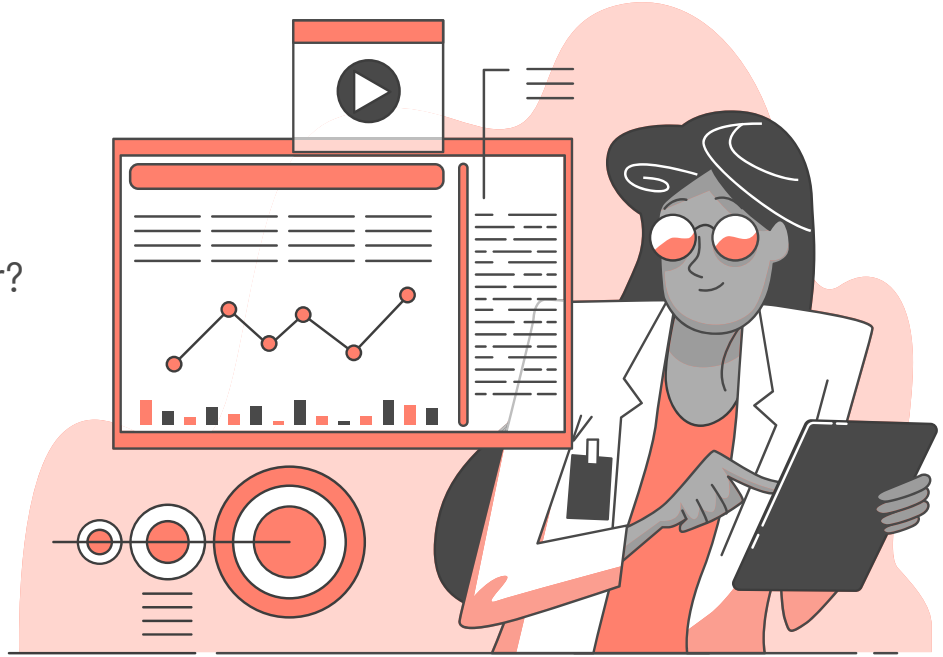
07

Why we are here?

The value of what we do
How to get the most benefit?
What to expect from us?

08

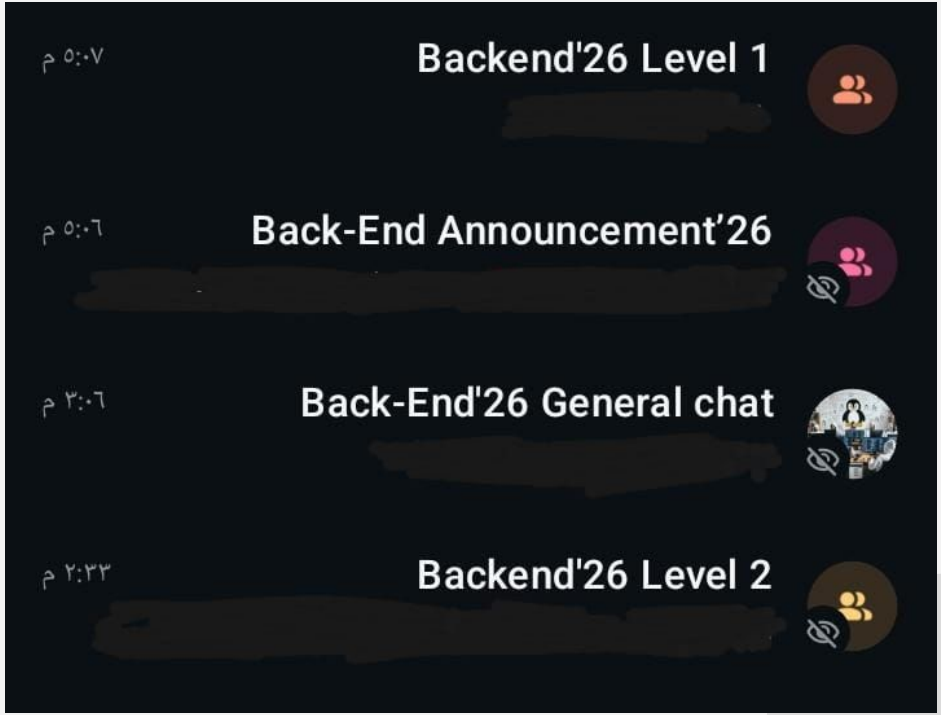
Party!!!!!



02 Tour

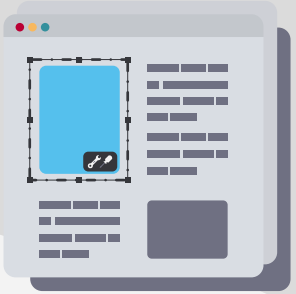
- Whatsapp Groups
- Github
- Discord
- Google Sheet
- Forms

02 Tour: Whatsapp Groups



02

Tour: Github



backend-learning-hub-26 Private

Watch 0

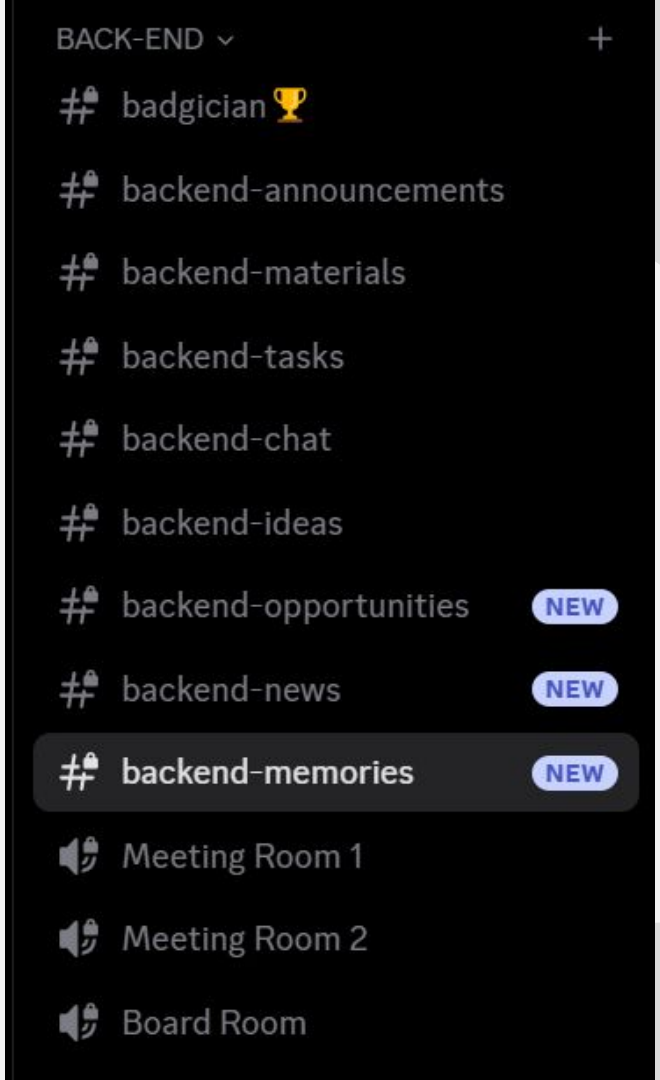
master | 1 Branch | 0 Tags | Go to file | Add file | Code

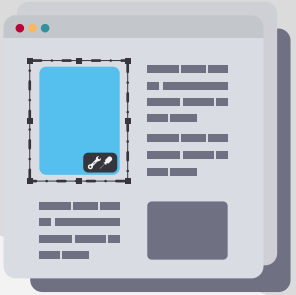
OMNIA-ibrahiim Merge branch 'master'of 8138725 · 10 minutes ago 30 Commits

.github/ISSUE_TEMPLATE	update(issues): Added issue templates	last week
Level-1	added session 2 folders in both levels	10 minutes ago
Level-2	added session 2 folders in both levels	10 minutes ago
More	docs(contribution): Edited the issue structure	last week
Welcome!	added two readme {contributing, how to get started}	last week
.gitignore	Extra structure edits + readme edit	last week
README.md	edite dthe readme	last week

README

Tour: Discord

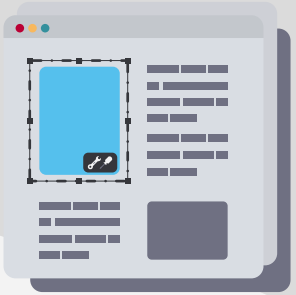




02 Tour: Sheet

A	B	C	D
	Important Links		
	Link	Description	
	https://github.com/Open-Source-Community	Open source community in github	
	https://github.com/Open-Source-Community/backend-learning-hub-26	Backend learning hub repo (on osc)	
	https://forms.gle/RRSP8fFHmbqVwzhW9	Feedback Form	
		ideas Form	
	https://chat.whatsapp.com/GOtOUVA9DjmKbkxFE8IrTR	Backend Genral chat Whatsapp Group	
	https://chat.whatsapp.com/IA3AuYf79JO36haYQBiWOt	Backend Announcment chat Whatsapp Group	
	https://www.facebook.com/groups/osccommunitytalk	OSC Community talk Facebook	
	https://www.facebook.com/groups/osccrew	OSC Crew Facebook	
	https://discord.gg/bBjmuBpn	Discord Channel	
	https://drive.google.com/drive/folders/1z8mHX6u	Stickers Drive	
	Board		

Tour: Forms



- *Feedback Form*
- *Excuses Form*
- *Attendance Form*

Table of contents

~~01~~

~~Recap~~

~~02~~

~~Tour in our Resources~~

03

What is Backend?

04

Backend Engineer Mindset

05

Languages & Tools Overview

06

JavaScript Deep Dive

03

What is Backend

What is Backend?

Definition:

- The backend = logic, data, and communication layer of any application.
- Handles requests, processes business logic, interacts with the database.

What is Backend?

Responsibilities

- Receive → Process → Store → Respond.

Backend vs Frontend

- Frontend: What the user sees.
- Backend: What makes it work.

What is Backend?

Client–Server Communication

- Client sends a request.
- Server processes it.
- Server sends a response.

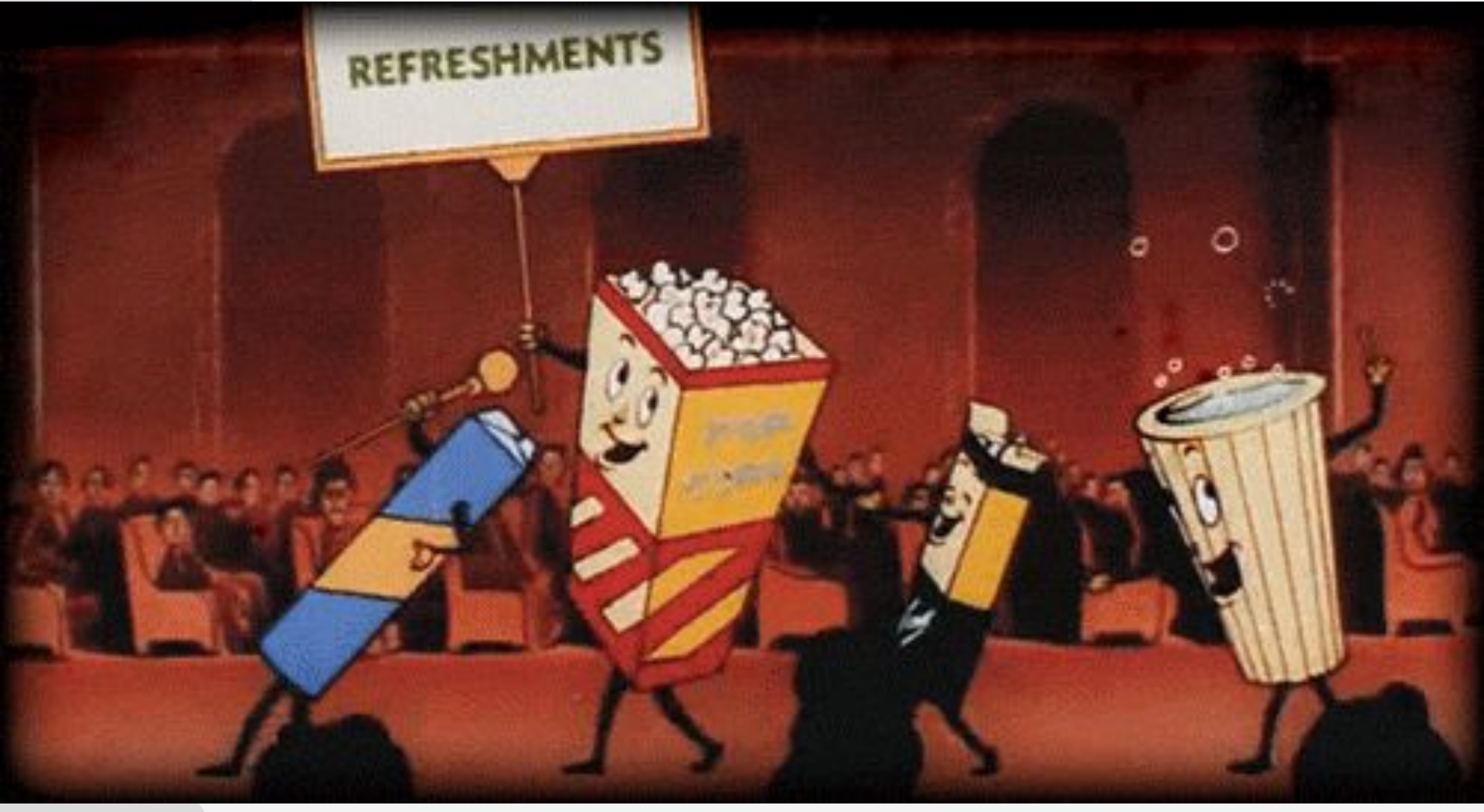
What is Backend?

Other types of Communication

- server-to-server communication
- asynchronous messaging patterns
- real-time communication

IT's Showtime

But who are the actors?


REFRESHMENTS

Thank you amazing Actors!

Table of contents

~~01~~

~~Recap~~

~~02~~

~~Tour in our Resources~~

~~03~~

~~What is Backend?~~

04

Backend Engineer Mindset

05

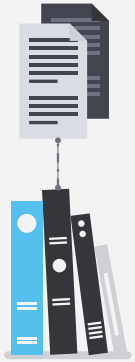
Languages & Tools Overview

06

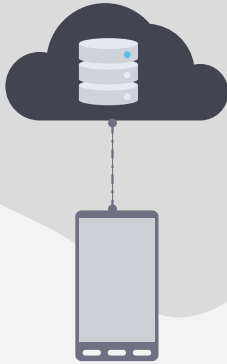
JavaScript Deep Dive

04

Backend Engineer Mindset



BackendEngineer Mindset



How backend engineers think

- Logic-oriented and detail-driven.
- Understand data flow & system behavior.
- Predict edge cases & failure points.

Core Responsibilities

- API design
- Database structure
- Security
- Performance

Attitude

- Debug-first mindset
- Patient → Analytical → Problem solver
- Ability to work with ambiguity

05 Backend Tools

Common Backend Languages

python

_Used for its simplicity and vast ecosystem

_Django, Flask frameworks

C#/.NET

_Microsoft's ecosystem with strong tooling

_Cross-platform with .NET Core

Java

_Spring Boot framework is widely used

_Excellent for large-scale, mission-critical systems

JavaScript/Node.js

_Allows using JavaScript on both frontend and backend.
_Non-blocking I/O makes it efficient for real-time applications
_Huge npm ecosystem

Go (Golang)

_Built by Google for high performance and concurrency

_Fast compilation and execution

PHP

_Powers much of the web (WordPress, Laravel)
_Easy to deploy and host

Ruby

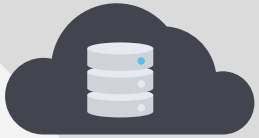
_Known for Ruby on Rails framework

_Developer-friendly with "convention over configuration"

INSIDE OUT 2
©2023 Disney/Pixar

TERMS ALERT

Languages



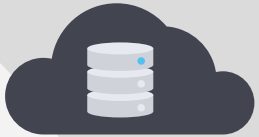
- **What it is:** The syntax and rules for writing code. The vocabulary and grammar.
- **Examples:** JavaScript , C++ , TypeScript
- **Think of it as:** The actual words and grammar rules - like English, Spanish, French.
- **Example:**

```
// JavaScript language syntax:
let name = "John";
if (name === "John")
  console.log("Hello");
```

- **Key point:** A language is just the rules for writing code. It doesn't DO anything by itself - it needs something to run it.

TERMS ALERT

Runtime Environment

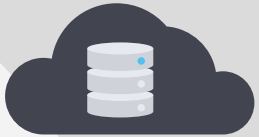


- **What it is:** Everything your code needs to actually RUN. The "stage" where your code performs.
- **Components:**
  - Engine/Interpreter (executes your code)
  - Built-in APIs (tools your code can use)
  - Memory management
  - Event loop (for async operations)

- **Examples:**
  - Browser (Chrome, Firefox) - runtime for JavaScript on web pages
  - Node.js - runtime for JavaScript on servers/computers
  - Python Interpreter - runtime for Python code
  - JVM (Java Virtual Machine) - runtime for Java code

TERMS ALERT

Runtime Environment



- **Think of it as:** A theater with a stage, props, lighting - everything needed for a performance.
- **Example:**

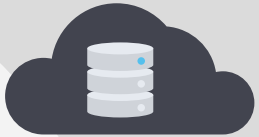
```
// In BROWSER runtime:
document.getElementById('btn'); // Works - browser provides 'document'
fs.readFile('file.txt'); //  Doesn't exist in browser

// In NODE.JS runtime:
fs.readFile('file.txt'); //  Works - Node provides 'fs'
document.getElementById('btn'); //  Doesn't exist in Node
```

- **Key point:** Same language (JavaScript), different runtimes = different capabilities.

TERMS ALERT

Framework



- **What it is:** Pre-written code that provides structure and tools for building applications. It makes decisions for you about architecture.
- **Characteristics:**
  - Opinionated - tells you how to organize code
  - Inversion of Control - YOU call libraries, FRAMEWORK calls your code
  - Provides structure, patterns, and best practices
- **Examples:**
  - React - framework for building UIs
  - Angular - complete framework for web apps
  - NestJS - framework for Node.js backend
  - Django - framework for Python web apps
  - Spring - framework for Java apps
- **Think of it as:** A pre-built house frame. Walls, roof structure already there - you just add finishing touches.
- **Key point:** Framework = someone built structure for you, you fill in the details.

TERMS Quest!

What is the difference between a framework and a library?

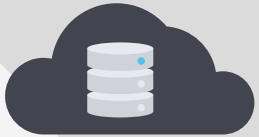
TERMS ALERT

Ecosystem

- An **ecosystem** is the entire interconnected world of tools, resources, people, and infrastructure that grows around a language or platform. It's everything that makes development possible and productive.
- **Think of it like a city:**
  - Language = the main language spoken
  - But a city needs: roads, stores, hospitals, schools, communities, laws, services
  - All of these working together = the ecosystem
- **How Ecosystem Affects You**
  - Strong Ecosystem (JavaScript, Python):
    - Problem? Someone already solved it - use a package
    - Stuck? Tons of Stack Overflow answers
    - Learning? Thousands of free tutorials
    - Jobs? Lots of opportunities
    - Deployment? Easy platforms available

TERMS ALERT

Ecosystem



JavaScript Ecosystem (Very Strong ★★★★★)

```
Package Manager:  npm (2+ million packages)
Frameworks:       React, Vue, Angular, NestJS, Next.js, Express
Build Tools:      Webpack, Vite, Rollup, Parcel
Testing:          Jest, Cypress, Playwright
Community:        HUGE - Stack Overflow, Reddit, Discord
Learning:         Tons of tutorials, courses, documentation
Jobs:             Millions of jobs worldwide
Deployment:       Vercel, Netlify, Heroku, AWS
```

Why strong:

- Largest package repository (npm)
- Used for frontend, backend, mobile, desktop
- Massive community
- Constantly evolving
- Easy to get help

Table of contents

~~01~~ ~~Recap~~

~~02~~ ~~Tour in our Resources~~

~~03~~ ~~What is Backend?~~

~~04~~ ~~Backend Engineer Mindset~~

~~05~~ ~~Languages & Tools Overview~~

06 JavaScript Deep Dive

06

JavaScrip
Deep Dive

What is JavaScript

- JavaScript is a high-level, interpreted programming language
- primarily known for enabling interactive and dynamic content on web pages.
- It is one of the three core technologies of the World Wide Web, alongside HTML (which structures content) and CSS (which styles content).

Where can we write and run JavaScript

Runtime Environments for Js

Browser

What is a Browser?

- a browser is an application

- it's software installed on your computer (like Chrome, Firefox, Safari, Edge).

Runtime Environments for Js

How JavaScript Runs in the Browser

1. The JavaScript Engine

Inside every browser, there's a JavaScript engine - a program that reads and executes JavaScript code:

- Chrome/Edge → V8 engine
- Firefox → SpiderMonkey
- Safari → JavaScriptCore

Think of the engine like a translator that converts your JavaScript code into machine code (instructions your computer's processor understands).

Runtime Environments for Js

2. The Process:

Your JS code → JS Engine reads it → Converts to machine code → CPU executes it

When you visit a website:

1. Browser downloads HTML, CSS, and JavaScript files
2. Browser reads the HTML and builds the page
3. When it encounters <script> tags, the JS engine executes that code
4. JavaScript can manipulate the page (change colors, respond to clicks, etc.)

Runtime Environments for Js

What Actually Happens When You Visit a Website

Step by step:

1. You type: www.facebook.com
2. Server sends you files:
   a. - index.html (plain text HTML)
   b. - styles.css (plain text CSS)
   c. - app.js (plain text JavaScript)
3. Your browser receives JavaScript as TEXT
4. The engine converts it to machine code RIGHT NOW, on your computer
5. Then it runs

Why the server does not send the Machine code ?

Runtime Environments for Js

Nodejs

- Node.js is JavaScript running OUTSIDE the browser.
- It's a different runtime environment.
- **Key Difference:**
  - Browser runtime = JS Engine + Web APIs (for web pages)
  - Node.js runtime = JS Engine (V8) + Node APIs (for servers/computers)

Game!

من سيربح المليون

Mohsens Edition!

1_ استعين بصديق

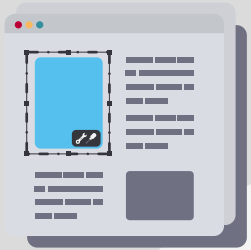
2_ حذف اجابتين

3_ تصويت الجمهور

theory mode: completed!

Time to roll up our sleeves and dive into the fun part

the practical work begins now!



Thanks!